Modelagem e Implementação do Banco de Dados

Sistema de vendas de um supermercado - Parte Prática

Aluno: Denilson Almeida Rocha | Curso: Análise e Desenvolvimento de Sistemas - UniFECAF

1. Visão geral

Este documento apresenta a modelagem do banco de dados definida a partir do levantamento de requisitos (Parte Teórica), em três níveis: o **modelo conceitual** (entidades e relacionamentos), o **modelo lógico** (tabelas, atributos, chaves e relacionamentos) e o **modelo físico** (script SQL compatível com MySQL 8.0.16 ou superior). O banco guarda produtos, clientes, colaboradores e vendas de forma estruturada, o que permite conhecer as preferências de compra de cada cliente e apoiar ações de fidelização.

2. Entidades do modelo

Tabela	Função	Obrigatória no enunciado
tbl_clientes	Cadastro opcional de clientes, identificados pelo CPF, com consentimento de ofertas e saldo de pontos.	Sim
tbl_produtos	Produtos à venda, com preço, estoque, estoque mínimo, categoria e fornecedor.	Sim
tbl_colaboradores	Funcionários do supermercado; o operador de caixa fica registrado em cada venda.	Sim
tbl_vendas	Cabeçalho de cada venda: data e hora, caixa, operador, cliente (opcional), total e status.	Sim
tbl_itens_venda	Produtos de cada venda, com quantidade, preço praticado e desconto.	Complementar
tbl_pagamentos_venda	Formas de pagamento usadas em cada venda, permitindo pagamento dividido.	Complementar
tbl_formas_pagamento	Lista de formas de pagamento aceitas (dinheiro, débito, crédito, Pix, vale).	Complementar
tbl_categorias	Categorias dos produtos (mercearia, bebidas, limpeza etc.).	Complementar
tbl_fornecedores	Fornecedor de cada produto.	Complementar

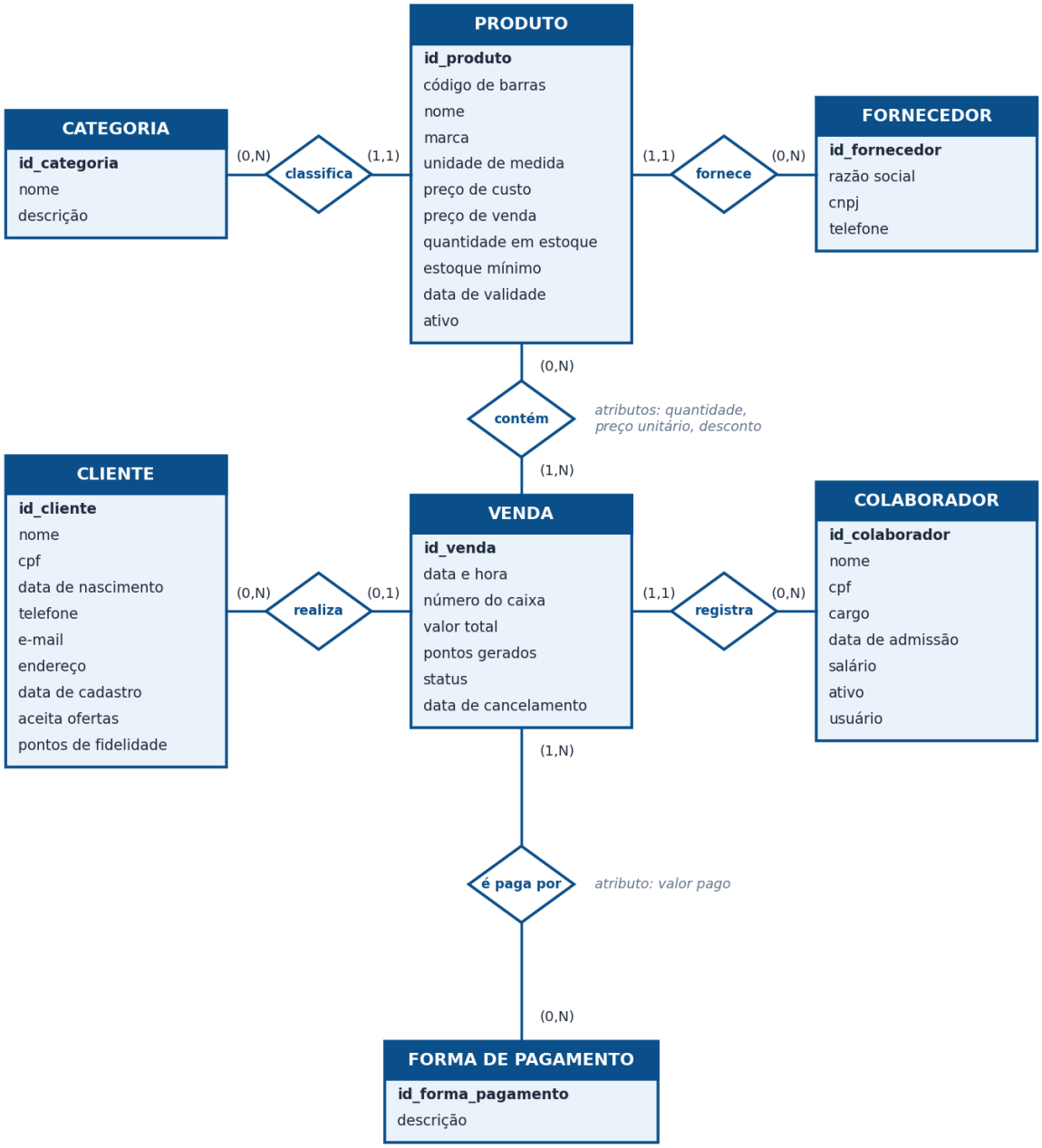
As quatro tabelas obrigatórias foram mantidas com os nomes sugeridos. As cinco complementares surgiram das respostas do levantamento: uma venda tem vários produtos (P8), pode ser paga de mais de uma forma (P9), e os produtos têm categoria e fornecedor (P3, P4).

Ordem de execução do script

- Executar o arquivo `script_supermercado.sql` inteiro no MySQL Workbench (ou na linha de comando) em um servidor MySQL 8.0.16 ou superior, que é a versão a partir da qual as restrições CHECK passam a ser aplicadas.
- O script cria o banco `db_supermercado`, as tabelas, os triggers, os dados de exemplo e as consultas de fidelização, nessa ordem.
- As seções 5 e 6 deste documento trazem o código completo e a conferência dos dados de exemplo.

3. Modelo Conceitual

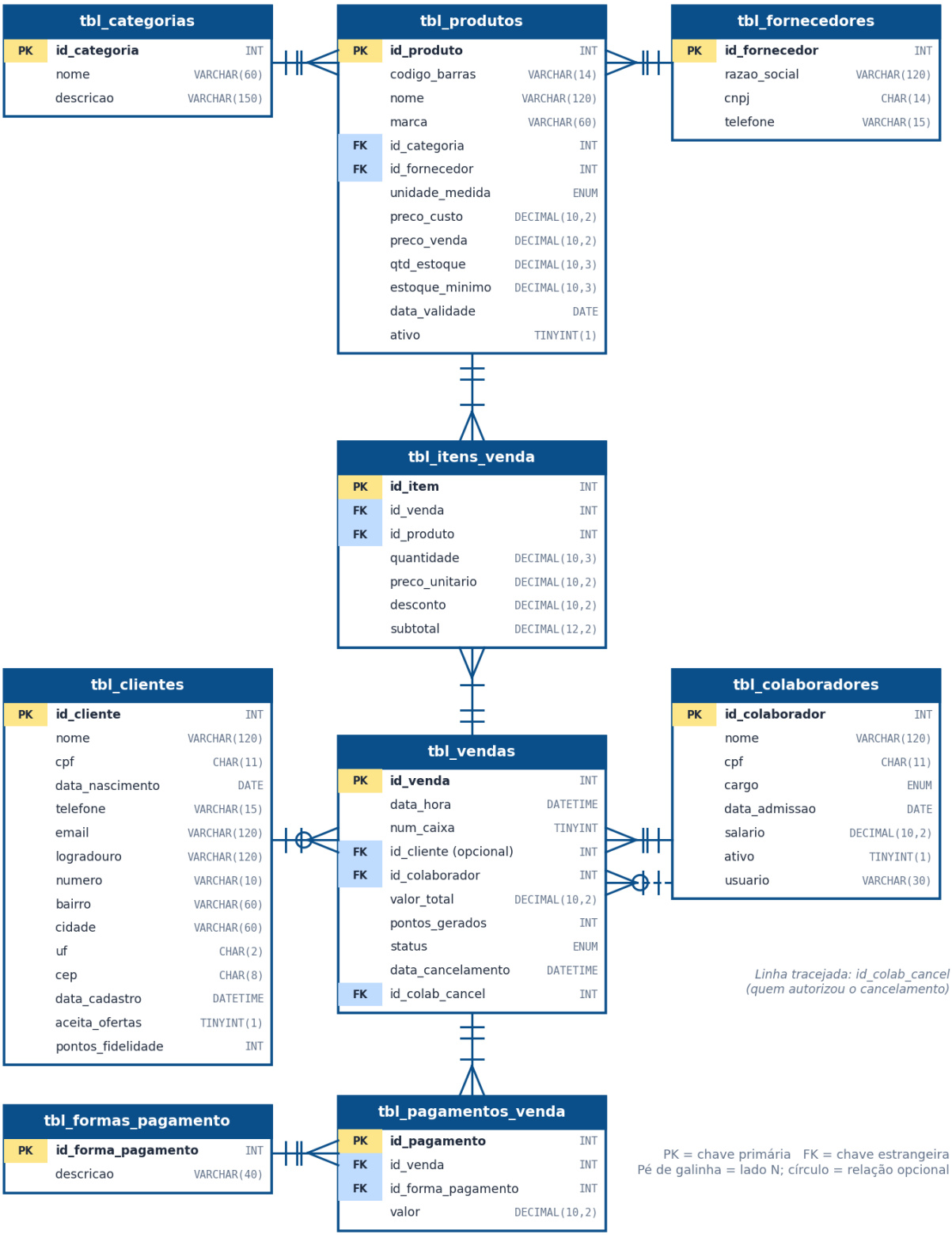
Diagrama entidade-relacionamento com os atributos de cada entidade e a cardinalidade no formato (mínimo, máximo). O relacionamento *contém* entre venda e produto é muito-para-muitos e carrega os atributos quantidade, preço unitário e desconto; o relacionamento *é paga por* entre venda e forma de pagamento também é muito-para-muitos e carrega o valor pago.



Identificador em negrito. Notação de cardinalidade: (mínimo, máximo).

4. Modelo Lógico

Tabelas, atributos com tipos de dados, chaves primárias (PK) e estrangeiras (FK). Os relacionamentos muitos-para-muitos do modelo conceitual foram resolvidos pelas tabelas *tbl_itens_venda* e *tbl_pagamentos_venda*.



4.1 Relacionamentos

Relacionamento	Card.	Chave estrangeira	Regra
tbl_categorias e tbl_produtos	1:N	tbl_produtos.id_categoria	Todo produto pertence a uma categoria.
tbl_fornecedores e tbl_produtos	1:N	tbl_produtos.id_fornecedor	Todo produto tem um fornecedor.
tbl_clientes e tbl_vendas	1:N	tbl_vendas.id_cliente (aceita NULL)	Venda sem CPF é permitida; cliente identificado acumula pontos.
tbl_colaboradores e tbl_vendas	1:N	tbl_vendas.id_colaborador e id_colab_cancel	Registra o operador da venda e quem autorizou um cancelamento.
tbl_vendas e tbl_itens_venda	1:N	tbl_itens_venda.id_venda	Uma venda tem um ou mais itens.
tbl_produtos e tbl_itens_venda	1:N	tbl_itens_venda.id_produto	Resolve o N:M entre venda e produto.
tbl_vendas e tbl_pagamentos_venda	1:N	tbl_pagamentos_venda.id_venda	Uma venda pode ter vários pagamentos.
tbl_formas_pagamento e tbl_pagamentos_venda	1:N	tbl_pagamentos_venda.id_forma_pagamento	Resolve o N:M entre venda e forma de pagamento.

4.2 Decisões de modelagem

- **Preço guardado no item:** *preco_unitario* copia o preço do produto no momento da venda, preservando o histórico quando o preço mudar.
- **Um produto por linha na venda:** a restrição única (id_venda, id_produto) evita linhas repetidas; para mais unidades, aumenta-se a quantidade.
- **Cliente opcional:** *id_cliente* aceita NULL, pois o cadastro não é obrigatório para comprar (P5).
- **Vendas nunca são apagadas:** o cancelamento muda o *status* para CANCELADA e registra data e responsável, e as chaves estrangeiras impedem exclusões que quebrem o histórico (P10).
- **Pagamentos em tabela própria:** permitem dividir uma compra entre dinheiro, cartão e Pix (P9).
- **Quantidades decimais:** *quantidade* e *qtd_estoque* usam três casas decimais para produtos vendidos por quilo.
- **Normalização (3FN):** categorias, fornecedores e formas de pagamento ficam em tabelas próprias para não repetir textos. *valor_total*, *pontos_gerados* e *subtotal* são dados derivados, mantidos de propósito para consulta rápida e para preservar o valor da época; os triggers e a coluna calculada garantem que fiquem consistentes.
- **LGPD:** o campo *aceita_ofertas* registra o consentimento para contato de marketing (P5, P12).
- **Limitação conhecida:** o cancelamento de uma venda não devolve estoque nem estorna pontos automaticamente; isso fica como evolução, via aplicação ou novo trigger.

5. Modelo Físico (código SQL para MySQL)

O script a seguir é o mesmo do arquivo `script_supermercado.sql`, com comentários explicando cada tabela e regra.

```
-- ========================================================================
-- SISTEMA DE VENDAS DE SUPERMERCADO - Modelo Físico (MySQL 8.0.16+)
-- Autor: Denilson Almeida Rocha | UniFECAF - Análise e Desenv. de Sistemas
-- ========================================================================
CREATE DATABASE IF NOT EXISTS db_supermercado
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE db_supermercado;

-- ------------------------------------------------------------------------
-- 1. Tabelas de apoio ao cadastro de produtos
-- ------------------------------------------------------------------------
CREATE TABLE tbl_categorias (
  id_categoria  INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome          VARCHAR(60)  NOT NULL,
  descricao     VARCHAR(150) NULL,
  CONSTRAINT pk_categorias PRIMARY KEY (id_categoria),
  CONSTRAINT uq_categorias_nome UNIQUE (nome)
) ENGINE=InnoDB;

CREATE TABLE tbl_fornecedores (
  id_fornecedor INT UNSIGNED NOT NULL AUTO_INCREMENT,
  razao_social  VARCHAR(120) NOT NULL,
  cnpj          CHAR(14)     NOT NULL,   -- somente números
  telefone      VARCHAR(15)  NULL,
  CONSTRAINT pk_fornecedores PRIMARY KEY (id_fornecedor),
  CONSTRAINT uq_fornecedores_cnpj UNIQUE (cnpj)
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- 2. tbl_produtos: itens vendidos e controle de estoque
-- ------------------------------------------------------------------------
CREATE TABLE tbl_produtos (
  id_produto     INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  codigo_barras  VARCHAR(14)   NOT NULL,
  nome           VARCHAR(120)  NOT NULL,
  marca          VARCHAR(60)   NULL,
  id_categoria   INT UNSIGNED  NOT NULL,
  id_fornecedor  INT UNSIGNED  NOT NULL,
  unidade_medida ENUM('UN','KG','L') NOT NULL DEFAULT 'UN',
  preco_custo    DECIMAL(10,2) NOT NULL,
  preco_venda    DECIMAL(10,2) NOT NULL,
  qtd_estoque    DECIMAL(10,3) NOT NULL DEFAULT 0,  -- 3 casas: produtos por kg
  estoque_minimo DECIMAL(10,3) NOT NULL DEFAULT 0,
  data_validade  DATE          NULL,                -- só para perecíveis
  ativo          TINYINT(1)    NOT NULL DEFAULT 1,  -- 0 = fora de linha
  CONSTRAINT pk_produtos PRIMARY KEY (id_produto),
  CONSTRAINT uq_produtos_barras UNIQUE (codigo_barras),
  CONSTRAINT fk_produtos_categoria FOREIGN KEY (id_categoria)
    REFERENCES tbl_categorias (id_categoria),
  CONSTRAINT fk_produtos_fornecedor FOREIGN KEY (id_fornecedor)
    REFERENCES tbl_fornecedores (id_fornecedor),
  CONSTRAINT chk_produtos_precos CHECK (preco_custo >= 0 AND preco_venda > 0),
  CONSTRAINT chk_produtos_estoque CHECK (qtd_estoque >= 0 AND estoque_minimo >= 0)
) ENGINE=InnoDB;

-- ------------------------------------------------------------------------
-- 3. tbl_clientes: cadastro opcional, identificado pelo CPF
-- ------------------------------------------------------------------------
CREATE TABLE tbl_clientes (
  id_cliente       INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome             VARCHAR(120) NOT NULL,
  cpf              CHAR(11)     NOT NULL,          -- somente números
  data_nascimento  DATE         NULL,
  telefone         VARCHAR(15)  NULL,
  email            VARCHAR(120) NULL,
  logradouro       VARCHAR(120) NULL,
  numero           VARCHAR(10)  NULL,
  bairro           VARCHAR(60)  NULL,
  cidade           VARCHAR(60)  NULL,
  uf               CHAR(2)      NULL,
  cep              CHAR(8)      NULL,
  data_cadastro    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  aceita_ofertas   TINYINT(1)   NOT NULL DEFAULT 0,  -- consentimento (LGPD)
  pontos_fidelidade INT         NOT NULL DEFAULT 0,  -- saldo do programa
  CONSTRAINT pk_clientes PRIMARY KEY (id_cliente),
  CONSTRAINT uq_clientes_cpf UNIQUE (cpf),
  CONSTRAINT chk_clientes_cpf CHECK (cpf REGEXP '^[0-9]{11}$'),
  CONSTRAINT chk_clientes_pontos CHECK (pontos_fidelidade >= 0)
) ENGINE=InnoDB;
```

```
-- ---------------------------------------------------------------------
-- 4. tbl_colaboradores: funcionários (operadores de caixa, gerentes...)
-- ---------------------------------------------------------------------
CREATE TABLE tbl_colaboradores (
  id_colaborador INT UNSIGNED NOT NULL AUTO_INCREMENT,
  nome           VARCHAR(120) NOT NULL,
  cpf            CHAR(11)     NOT NULL,
  cargo          ENUM('OPERADOR_CAIXA','GERENTE','REPOSITOR','ACOUGUEIRO') NOT NULL,
  data_admissao  DATE         NOT NULL,
  salario        DECIMAL(10,2) NOT NULL,
  ativo          TINYINT(1)   NOT NULL DEFAULT 1,   -- 0 = desligado
  usuario        VARCHAR(30)  NOT NULL,             -- login (senha fica no app)
  CONSTRAINT pk_colaboradores PRIMARY KEY (id_colaborador),
  CONSTRAINT uq_colaboradores_cpf UNIQUE (cpf),
  CONSTRAINT uq_colaboradores_usuario UNIQUE (usuario),
  CONSTRAINT chk_colaboradores_salario CHECK (salario > 0)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 5. tbl_formas_pagamento: dinheiro, débito, crédito, Pix, vale...
-- ---------------------------------------------------------------------
CREATE TABLE tbl_formas_pagamento (
  id_forma_pagamento INT UNSIGNED NOT NULL AUTO_INCREMENT,
  descricao          VARCHAR(40)  NOT NULL,
  CONSTRAINT pk_formas_pagamento PRIMARY KEY (id_forma_pagamento),
  CONSTRAINT uq_formas_pagamento UNIQUE (descricao)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 6. tbl_vendas: cabeçalho da venda (id_cliente é opcional)
-- ---------------------------------------------------------------------
CREATE TABLE tbl_vendas (
  id_venda          INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  data_hora         DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  num_caixa         TINYINT UNSIGNED NOT NULL,
  id_cliente        INT UNSIGNED  NULL,            -- NULL = cliente não identificado
  id_colaborador    INT UNSIGNED  NOT NULL,        -- operador que fez a venda
  valor_total       DECIMAL(10,2) NOT NULL DEFAULT 0,  -- atualizado pelo trigger
  pontos_gerados    INT           NOT NULL DEFAULT 0,  -- atualizado pelo trigger
  status            ENUM('CONCLUIDA','CANCELADA') NOT NULL DEFAULT 'CONCLUIDA',
  data_cancelamento DATETIME      NULL,
  id_colab_cancel   INT UNSIGNED  NULL,            -- quem autorizou o cancelamento
  CONSTRAINT pk_vendas PRIMARY KEY (id_venda),
  CONSTRAINT fk_vendas_cliente FOREIGN KEY (id_cliente)
    REFERENCES tbl_clientes (id_cliente),
  CONSTRAINT fk_vendas_colaborador FOREIGN KEY (id_colaborador)
    REFERENCES tbl_colaboradores (id_colaborador),
  CONSTRAINT fk_vendas_colab_cancel FOREIGN KEY (id_colab_cancel)
    REFERENCES tbl_colaboradores (id_colaborador),
  CONSTRAINT chk_vendas_valor CHECK (valor_total >= 0),
  CONSTRAINT chk_vendas_cancel CHECK (
    (status = 'CANCELADA' AND data_cancelamento IS NOT NULL) OR
    (status = 'CONCLUIDA' AND data_cancelamento IS NULL)),
  INDEX idx_vendas_cliente_data (id_cliente, data_hora),  -- histórico por cliente
  INDEX idx_vendas_data (data_hora)                       -- relatórios por período
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 7. tbl_itens_venda: produtos de cada venda (resolve o N:M venda x produto)
--    Guarda o preço praticado, pois o preço do produto muda com o tempo.
-- ---------------------------------------------------------------------
CREATE TABLE tbl_itens_venda (
  id_item        INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  id_venda       INT UNSIGNED  NOT NULL,
  id_produto     INT UNSIGNED  NOT NULL,
  quantidade     DECIMAL(10,3) NOT NULL,
  preco_unitario DECIMAL(10,2) NOT NULL,
  desconto       DECIMAL(10,2) NOT NULL DEFAULT 0,
  subtotal       DECIMAL(12,2) AS (ROUND(quantidade * preco_unitario - desconto, 2)) STORED,
  CONSTRAINT pk_itens_venda PRIMARY KEY (id_item),
  CONSTRAINT uq_itens_venda_produto UNIQUE (id_venda, id_produto),
  CONSTRAINT fk_itens_venda FOREIGN KEY (id_venda)
    REFERENCES tbl_vendas (id_venda),
  CONSTRAINT fk_itens_produto FOREIGN KEY (id_produto)
    REFERENCES tbl_produtos (id_produto),
  CONSTRAINT chk_itens_valores CHECK (
    quantidade > 0 AND preco_unitario > 0 AND desconto >= 0
    AND desconto <= quantidade * preco_unitario)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 8. tbl_pagamentos_venda: formas de pagamento de cada venda (N:M)
-- ---------------------------------------------------------------------
CREATE TABLE tbl_pagamentos_venda (
  id_pagamento       INT UNSIGNED  NOT NULL AUTO_INCREMENT,
```

```
  id_venda           INT UNSIGNED  NOT NULL,
  id_forma_pagamento INT UNSIGNED  NOT NULL,
  valor              DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_pagamentos_venda PRIMARY KEY (id_pagamento),
  CONSTRAINT fk_pagamentos_venda FOREIGN KEY (id_venda)
    REFERENCES tbl_vendas (id_venda),
  CONSTRAINT fk_pagamentos_forma FOREIGN KEY (id_forma_pagamento)
    REFERENCES tbl_formas_pagamento (id_forma_pagamento),
  CONSTRAINT chk_pagamentos_valor CHECK (valor > 0)
) ENGINE=InnoDB;

-- ---------------------------------------------------------------------
-- 9. Triggers: automatizam estoque, total da venda e pontos de fidelidade
-- ---------------------------------------------------------------------
DELIMITER $$

-- Ao inserir um item: baixa o estoque e soma o item no total da venda.
-- Os pontos são 1 por real gasto (parte inteira do total da venda).
CREATE TRIGGER trg_itens_venda_ai
AFTER INSERT ON tbl_itens_venda
FOR EACH ROW
BEGIN
  UPDATE tbl_produtos
     SET qtd_estoque = qtd_estoque - NEW.quantidade
   WHERE id_produto = NEW.id_produto;

  UPDATE tbl_vendas
     SET valor_total    = valor_total
                          + ROUND(NEW.quantidade * NEW.preco_unitario - NEW.desconto, 2),
         pontos_gerados = FLOOR(valor_total)
   WHERE id_venda = NEW.id_venda;
END$$

-- Quando os pontos da venda aumentam, credita a diferença ao cliente
-- (só se a venda tiver cliente identificado).
CREATE TRIGGER trg_vendas_au
AFTER UPDATE ON tbl_vendas
FOR EACH ROW
BEGIN
  IF NEW.id_cliente IS NOT NULL AND NEW.pontos_gerados <> OLD.pontos_gerados THEN
    UPDATE tbl_clientes
       SET pontos_fidelidade = pontos_fidelidade
                             + (NEW.pontos_gerados - OLD.pontos_gerados)
     WHERE id_cliente = NEW.id_cliente;
  END IF;
END$$

DELIMITER ;

-- ---------------------------------------------------------------------
-- 10. Dados de exemplo (para testar o banco)
-- ---------------------------------------------------------------------
INSERT INTO tbl_categorias (id_categoria, nome) VALUES
  (1,'Mercearia'), (2,'Hortifrúti'), (3,'Bebidas'), (4,'Limpeza'), (5,'Laticínios');

INSERT INTO tbl_fornecedores (id_fornecedor, razao_social, cnpj, telefone) VALUES
  (1,'Distribuidora Alimentos Brasil Ltda','11222333000181','1133334444'),
  (2,'Bebidas do Vale S.A.','22333444000172','1144445555'),
  (3,'Limpa Mais Produtos Ltda','33444555000163','1155556666');

INSERT INTO tbl_produtos (id_produto, codigo_barras, nome, marca, id_categoria, id_fornecedor,
  unidade_medida, preco_custo, preco_venda, qtd_estoque, estoque_minimo) VALUES
  (1,'7891000100103','Arroz Tipo 1 5kg','Bom Grão',1,1,'UN',18.50,24.90,120,20),
  (2,'7891000200204','Feijão Carioca 1kg','Bom Grão',1,1,'UN',6.20,8.99,200,30),
  (3,'7891000300305','Leite Integral 1L','Campo Azul',5,1,'UN',3.90,5.49,300,50),
  (4,'2000000000047','Banana Prata','Hortifruti Local',2,1,'KG',3.20,5.99,80,10),
  (5,'7891000500507','Refrigerante Cola 2L','Gás Cola',3,2,'UN',5.50,9.49,150,24),
  (6,'7891000600608','Detergente 500ml','Brilha',4,3,'UN',1.60,2.99,180,30);

INSERT INTO tbl_clientes (id_cliente, nome, cpf, telefone, email, cidade, uf,
  aceita_ofertas) VALUES
  (1,'Ana Paula Ferreira','52998224725','11988887777','ana.ferreira@email.com','São Paulo','SP',1),
  (2,'João Pedro Santos','11144477735','11977776666','joao.santos@email.com','São Paulo','SP',1);

INSERT INTO tbl_colaboradores (id_colaborador, nome, cpf, cargo, data_admissao, salario,
  usuario) VALUES
  (1,'Maria Souza','12345678909','OPERADOR_CAIXA','2024-03-10',2100.00,'maria.souza'),
  (2,'Carlos Lima','98765432100','GERENTE','2022-08-01',4800.00,'carlos.lima');

INSERT INTO tbl_formas_pagamento (id_forma_pagamento, descricao) VALUES
  (1,'Dinheiro'), (2,'Cartão de Débito'), (3,'Cartão de Crédito'), (4,'Pix'),
  (5,'Vale-Alimentação');

-- Vendas (valor_total e pontos começam em 0 e são calculados pelos triggers)
INSERT INTO tbl_vendas (id_venda, data_hora, num_caixa, id_cliente, id_colaborador) VALUES
```

```
  (1,'2026-09-28 10:15:00',2,1,1),
  (2,'2026-09-28 11:40:00',2,2,1),
  (3,'2026-09-29 18:05:00',1,NULL,1),   -- cliente não identificado
  (4,'2026-10-01 09:30:00',3,1,1);

INSERT INTO tbl_itens_venda (id_venda, id_produto, quantidade, preco_unitario, desconto) VALUES
  (1,1,1,24.90,0), (1,2,2,8.99,0), (1,3,3,5.49,0), (1,4,2.000,5.99,0),
  (2,5,2,9.49,0),  (2,6,3,2.99,0),
  (3,3,2,5.49,0),  (3,1,1,24.90,2.00),
  (4,3,4,5.49,0),  (4,4,1.000,5.99,0);

-- Pagamentos (a venda 1 foi dividida entre dinheiro e Pix)
INSERT INTO tbl_pagamentos_venda (id_venda, id_forma_pagamento, valor) VALUES
  (1,1,30.00), (1,4,41.33),
  (2,3,27.95),
  (3,2,33.88),
  (4,4,27.95);

-- ----------------------------------------------------------------------
-- 11. Consultas de apoio à fidelização de clientes
-- ----------------------------------------------------------------------

-- 11.1 Produtos mais comprados por cada cliente (base para ofertas direcionadas)
SELECT c.nome AS cliente, p.nome AS produto,
       SUM(i.quantidade) AS qtd_total,
       COUNT(DISTINCT v.id_venda) AS compras
  FROM tbl_vendas v
  JOIN tbl_clientes c     ON c.id_cliente = v.id_cliente
  JOIN tbl_itens_venda i  ON i.id_venda   = v.id_venda
  JOIN tbl_produtos p     ON p.id_produto = i.id_produto
 WHERE v.status = 'CONCLUIDA'
 GROUP BY c.id_cliente, c.nome, p.id_produto, p.nome
 ORDER BY c.nome, qtd_total DESC;

-- 11.2 Clientes que aceitam ofertas e não compram há mais de 60 dias
SELECT c.nome, c.email, c.telefone, MAX(v.data_hora) AS ultima_compra
  FROM tbl_clientes c
  JOIN tbl_vendas v ON v.id_cliente = c.id_cliente AND v.status = 'CONCLUIDA'
 WHERE c.aceita_ofertas = 1
 GROUP BY c.id_cliente, c.nome, c.email, c.telefone
HAVING MAX(v.data_hora) < DATE_SUB(NOW(), INTERVAL 60 DAY);

-- 11.3 Número de compras, ticket médio e saldo de pontos por cliente
SELECT c.nome, COUNT(*) AS compras,
       ROUND(AVG(v.valor_total), 2) AS ticket_medio,
       c.pontos_fidelidade
  FROM tbl_clientes c
  JOIN tbl_vendas v ON v.id_cliente = c.id_cliente AND v.status = 'CONCLUIDA'
 GROUP BY c.id_cliente, c.nome, c.pontos_fidelidade;

-- 11.4 Produtos ativos com estoque abaixo ou igual ao mínimo (reposição)
SELECT nome, qtd_estoque, estoque_minimo
  FROM tbl_produtos
 WHERE ativo = 1 AND qtd_estoque <= estoque_minimo;

-- 11.5 Conferência dos dados de exemplo (valores esperados nos comentários)
--      venda 1: 71.33 / 71 pts | venda 2: 27.95 / 27 | venda 3: 33.88 / 33 | venda 4: 27.95 / 27
SELECT id_venda, valor_total, pontos_gerados FROM tbl_vendas ORDER BY id_venda;
```

6. Conferência dos dados de exemplo

Depois de executar o script, os triggers devem calcular o total de cada venda, os pontos e o novo estoque. Os valores esperados abaixo foram calculados a partir dos dados inseridos.

id_venda	valor_total	pontos_gerados	Cliente
1	71.33	71	Ana Paula Ferreira
2	27.95	27	João Pedro Santos
3	33.88	33	não identificado
4	27.95	27	Ana Paula Ferreira

Saldo final de pontos: Ana Paula = 98 (71 + 27) e João Pedro = 27. Na consulta 11.3, o ticket médio da Ana Paula é 49.64 (duas compras) e o de João Pedro é 27.95. Estoque final: arroz 118, feijão 198, leite 291, banana 77, refrigerante 148 e detergente 177.